In [1]:
```
# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense
```

In [2]:
```
# 2. Load training and testing datasets
train_data = pd.read_csv("sign_mnist_train.csv")
test_data = pd.read_csv("sign_mnist_test.csv")

# 3. Separate images and labels
X_train = train_data.iloc[:, 1:].values
y_train = train_data.iloc[:, 0].values

X_test = test_data.iloc[:, 1:].values
y_test = test_data.iloc[:, 0].values

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (27455, 784)
Testing data: (7172, 784)
```

In [3]:
```
# 4. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 5. Reshape images to 28x28 pixels with 1 channel
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

In [6]:
```
# 6. Define the CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation="relu",
           input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
```

```
    Flatten(),
    Dense(128, activation="relu"),
    Dense(25, activation="softmax")
])

# Display model architecture
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_2 (Flatten)	(None, 5408)	0
dense_4 (Dense)	(None, 128)	692,352
dense_5 (Dense)	(None, 25)	3,225

Total params: 695,897 (2.65 MB)

Trainable params: 695,897 (2.65 MB)

Non-trainable params: 0 (0.00 B)

In [7]:
```
# 7. Compile the model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 8. Train the model
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=32,
```

```
    validation_split=0.1
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/5
773/773 ━━━━━━━━━━━━━━━━━━━━ 16s 18ms/step - accuracy: 0.6481 - loss: 1.2702 - val_accuracy: 0.9009 - val_loss: 0.4356
Epoch 2/5
773/773 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9551 - loss: 0.2194 - val_accuracy: 0.9920 - val_loss: 0.0906
Epoch 3/5
773/773 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9969 - loss: 0.0527 - val_accuracy: 0.9996 - val_loss: 0.0284
Epoch 4/5
773/773 ━━━━━━━━━━━━━━━━━━━━ 15s 19ms/step - accuracy: 0.9997 - loss: 0.0182 - val_accuracy: 0.9996 - val_loss: 0.0118
Epoch 5/5
773/773 ━━━━━━━━━━━━━━━━━━━━ 14s 18ms/step - accuracy: 1.0000 - loss: 0.0079 - val_accuracy: 1.0000 - val_loss: 0.0053
```

In [8]:
```
# 9. Evaluate using the separate test dataset
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")
```

```
225/225 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8074 - loss: 0.8409
Test Loss: 0.8408833146095276
Test Accuracy: 80.74456453323364 %
```

In [9]:
```
# 10. Plot training and validation accuracy and loss
plt.figure(figsize=(12, 4))

# Accuracy graph
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

# Loss graph
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
```

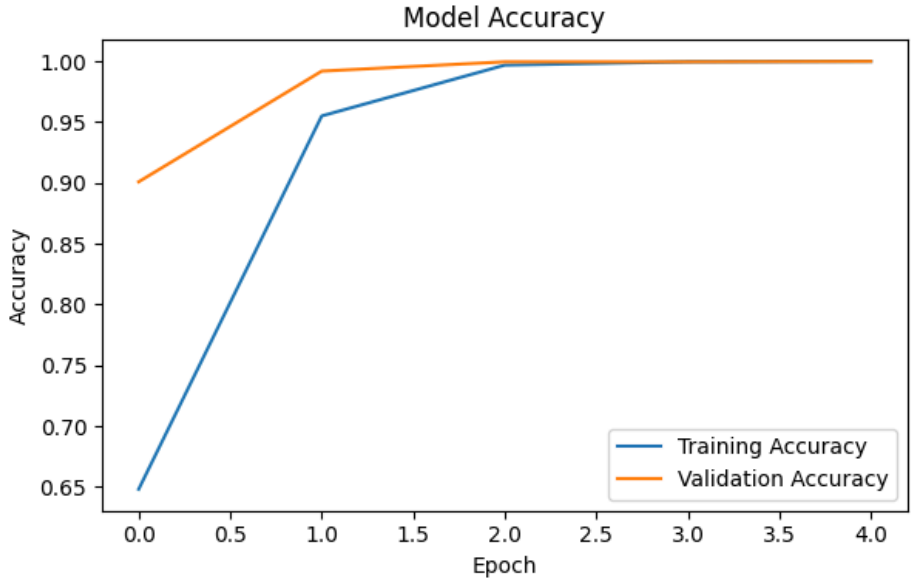
```
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

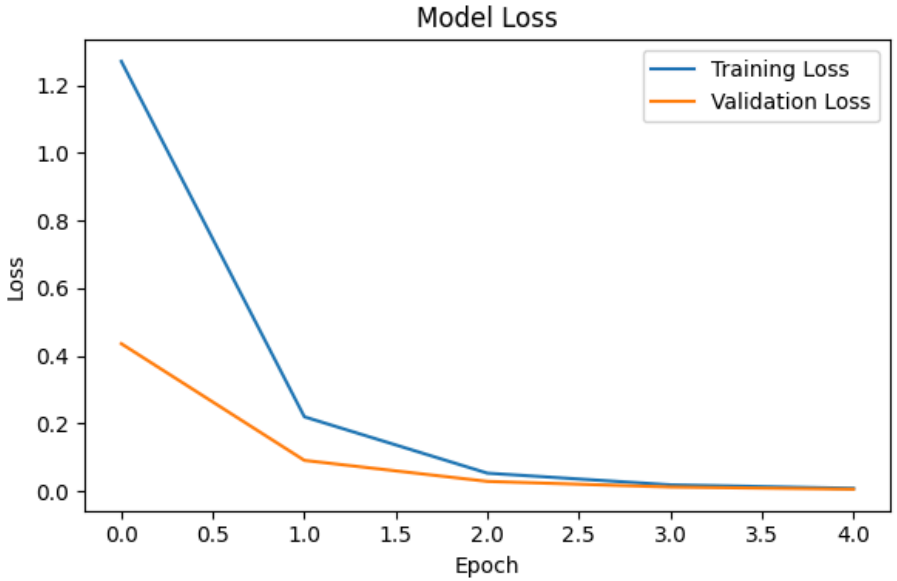
plt.tight_layout()
plt.show()

# 11. Predict a sample image
index = 0

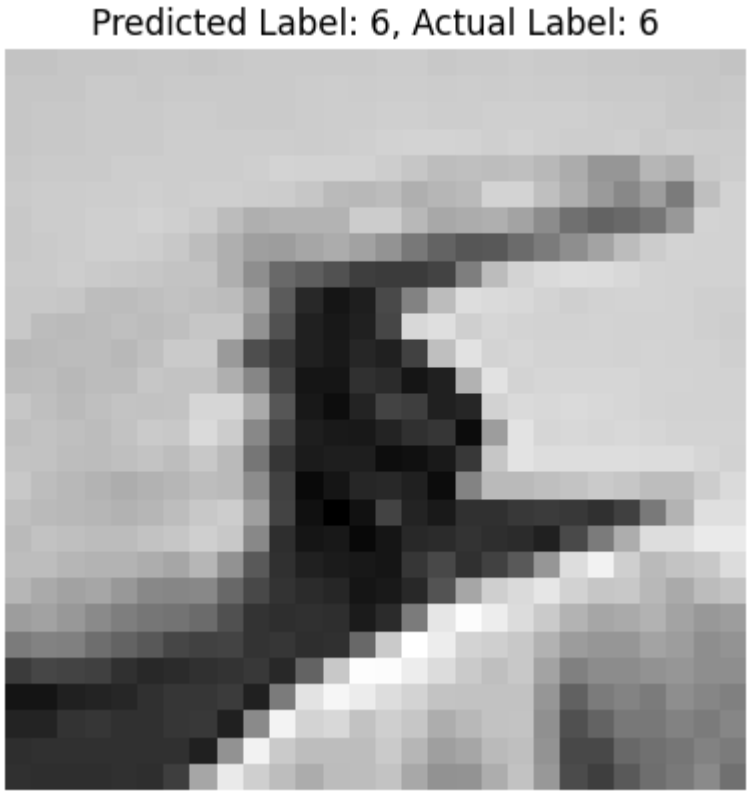
prediction = model.predict(X_test[index:index+1])
predicted_label = np.argmax(prediction)

plt.imshow(X_test[index].reshape(28, 28), cmap="gray")
plt.title(
    f"Predicted Label: {predicted_label}, Actual Label: {y_test[index]}"
)
plt.axis("off")
plt.show()
```






```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 158ms/step
```



In [ ]: